

MCP

Connect Claude Code to tools via MCP

Learn how to connect Claude Code to your tools with the Model Context Protocol.

Claude Code can connect to hundreds of external tools and data sources through the [**Model Context Protocol \(MCP\)**](#), an open source standard for AI-tool integrations. MCP servers give Claude Code access to your tools, databases, and APIs.

Connect a server when you find yourself copying data into chat from another tool, like an issue tracker or a monitoring dashboard. Once connected, Claude can read and act on that system directly instead of working from what you paste.

If you're connecting your first server, start with the [**MCP quickstart**](#) for a step-by-step walkthrough. This page is the full reference.

What you can do with MCP

With MCP servers connected, you can ask Claude Code to:

- **Implement features from issue trackers:** “Add the feature described in JIRA issue ENG-4521 and create a PR on GitHub.”
- **Analyze monitoring data:** “Check Sentry and Statsig to check the usage of the feature described in ENG-4521.”
- **Query databases:** “Find emails of 10 random users who used feature ENG-4521, based on our PostgreSQL database.”
- **Integrate designs:** “Update our standard email template based on the new Figma designs that were posted in Slack”
- **Automate workflows:** “Create Gmail drafts inviting these 10 users to a feedback session about the new feature.”
- **React to external events:** an MCP server can also act as a [**channel**](#) that pushes messages into your session, so Claude reacts to Telegram messages, Discord chats, or webhook events while you're away.

Find and build MCP servers

Browse reviewed connectors in the [Anthropic Directory](#). Directory connectors use the same MCP infrastructure as Claude Code, so you can add any remote server listed there with `claude mcp add`.

⚠️ Verify you trust each server before connecting it. Servers that fetch external content can expose you to [prompt injection risk](#).

To build your own server, see the [MCP server guide](#) for protocol fundamentals and the [Claude connector building docs](#) for authentication, testing, and Directory submission.

You can also have Claude scaffold a server for you with the official [mcp-server-dev plugin](#).

1 Install the plugin

In a Claude Code session, run:

```
/plugin install mcp-server-dev@claude-plugins-official
```

If Claude Code reports that the marketplace is not found, run `/plugin marketplace add anthropics/claude-plugins-official` first, then retry the install. Once installed, run `/reload-plugins` to activate it in the current session.

2 Run the build skill

```
/mcp-server-dev:build-mcp-server
```

Claude asks about your use case and scaffolds a remote HTTP or local stdio server.

Installing MCP servers

MCP servers can be configured in several ways depending on your needs:

Option 1: Add a remote HTTP server

HTTP servers are the recommended option for connecting to remote MCP servers. This is the most

widely supported transport for cloud-based services.

```
# Basic syntax
claude mcp add --transport http <name> <url>

# Real example: Connect to Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# Example with Bearer token
claude mcp add --transport http secure-api
https://api.example.com/mcp \
  --header "Authorization: Bearer your-token"
```

When configuring MCP servers via JSON in `.mcp.json`, `~/.claude.json`, or `claude mcp add-json`, the `type` field accepts `streamable-http` as an alias for `http`. The MCP specification uses the name `streamable-http` for this transport, so configurations copied from server documentation work without modification.

Option 2: Add a remote SSE server

 The SSE (Server-Sent Events) transport is deprecated. Use HTTP servers instead, where available.

```
# Basic syntax
claude mcp add --transport sse <name> <url>

# Real example: Connect to Asana
claude mcp add --transport sse asana https://mcp.asana.com/sse

# Example with authentication header
claude mcp add --transport sse private-api
https://api.company.com/sse \
  --header "X-API-Key: your-key-here"
```

Option 3: Add a local stdio server

Stdio servers run as local processes on your machine. They're ideal for tools that need direct system access or custom scripts.

Claude Code sets `CLAUDE_PROJECT_DIR` in the spawned server's environment to the project root, so your server can resolve project-relative paths without depending on the working directory. This is the same directory hooks receive in their `CLAUDE_PROJECT_DIR` variable. Read it from inside your server process, for example `process.env.CLAUDE_PROJECT_DIR` in Node or `os.environ["CLAUDE_PROJECT_DIR"]` in Python.

Your server can also call the MCP `roots/list` request, which returns the directory Claude Code was launched from.

This variable is set in the server's environment, not in Claude Code's own environment, so referencing it via `${VAR}` expansion in a project- or user-scoped `.mcp.json` `command` or `args` requires a default such as `${CLAUDE_PROJECT_DIR:-.}`. Plugin-provided MCP configurations substitute `${CLAUDE_PROJECT_DIR}` directly and don't need the default.

```
# Basic syntax
claude mcp add [options] <name> -- <command> [args...]

# Real example: Add Airtable server
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio
airtable \
  -- npx -y airtable-mcp-server
```

❗ Important: Separate server arguments with `--`

For stdio servers, the `--` (double dash) separates Claude's own options, such as `--transport`, `--env`, and `--scope`, from the command and arguments that run the server. Everything after `--` is passed to the server untouched.

For example:

- `claude mcp add --transport stdio myserver -- npx server` → runs `npx server`
- `claude mcp add --env KEY=value --transport stdio myserver -- python server.py --port 8080` → runs `python server.py --port 8080` with `KEY=value` in environment

Without `--`, Claude Code would try to parse the server's flags, like `--port` above, as its own options.

`--env` accepts multiple `KEY=value` pairs. If the server name comes directly after `--env`, the CLI reads the name as another pair and rejects it, so place at least one other option between `--env` and the server name, as in the examples above.

Option 4: Add a remote WebSocket server

WebSocket servers hold a persistent bidirectional connection, which suits remote MCP servers that

push events to Claude unprompted. Use HTTP instead when your server only responds to requests, since HTTP supports OAuth and the `claude mcp add --transport` flag, while WebSocket supports neither.

Configure WebSocket servers in `.mcp.json` or with `claude mcp add-json` :

```
claude mcp add-json events-server \  
  '{"type":"ws","url":"wss://mcp.example.com/socket","headers":  
  {"Authorization":"Bearer YOUR_TOKEN"}}'
```

The `type: "ws"` entry accepts the same `url` , `headers` , `headersHelper` , `timeout` , and `alwaysLoad` fields as `http` . Authentication is header-only, so pass a static token in `headers` or generate one at connect time with `headersHelper` . The `claude mcp add --transport` flag doesn't accept `ws` .

Managing your servers

Once configured, you can manage your MCP servers with these commands:

```
# List all configured servers  
claude mcp list  
  
# Get details for a specific server  
claude mcp get github  
  
# Remove a server  
claude mcp remove github  
  
# (within Claude Code) Check server status  
/mcp
```

Project-scoped servers from `.mcp.json` that are awaiting your approval appear in `claude mcp list` as `⏸ Pending approval` . Run `claude` interactively to review and approve them. `claude mcp get <name>` shows pending servers as `⏸ Pending approval` and rejected servers as `✗ Rejected` .

The `/mcp` panel shows the tool count next to each connected server and flags servers that advertise the tools capability but expose no tools.

If your request needs tools from a server that is still connecting in the background, Claude waits for that server before continuing. With **tool search** enabled, which is the default, the wait happens inside the `ToolSearch` call. In configurations without tool search, such as Vertex AI, a custom `ANTHROPIC_BASE_URL`, or `ENABLE_TOOL_SEARCH=false`, Claude uses the `WaitForMcpServers` tool instead.

The server name `workspace` is reserved for internal use. If your configuration defines a server with that name, Claude Code skips it at load time and shows a warning asking you to rename it.

Dynamic tool updates

Claude Code supports MCP `list_changed` notifications, allowing MCP servers to dynamically update their available tools, prompts, and resources without requiring you to disconnect and reconnect. When an MCP server sends a `list_changed` notification, Claude Code automatically refreshes the available capabilities from that server.

Automatic reconnection

If an HTTP or SSE server disconnects mid-session, Claude Code automatically reconnects with exponential backoff: up to five attempts, starting at a one-second delay and doubling each time. The server appears as pending in `/mcp` while reconnection is in progress. After five failed attempts the server is marked as failed and you can retry manually from `/mcp`. Stdio servers are local processes and are not reconnected automatically.

The same backoff applies when an HTTP or SSE server fails its initial connection at startup. As of v2.1.121, Claude Code retries the initial connection up to three times on transient errors such as a 5xx response, a connection refused, or a timeout, then marks the server as failed if it still can't connect. Authentication and not-found errors are not retried because they require a configuration change to resolve.

As of v2.1.191, the capability discovery requests that run after a successful connection, such as `tools/list`, `prompts/list`, and `resources/list`, also retry transient network and server errors up to three times with short backoff. Authentication errors, 4xx responses, and request timeouts are not retried.

Push messages with channels

An MCP server can also push messages directly into your session so Claude can react to external events like CI results, monitoring alerts, or chat messages. To enable this, your server declares the

`claude/channel` capability and you opt it in with the `--channels` flag at startup. See [Channels](#) to use an officially supported channel, or [Channels reference](#) to build your own.

💡 Tips:

- Use the `--scope` flag to specify where the configuration is stored:
 - `local` (default): available only to you in the current project. Older versions called this scope `project`
 - `project` : shared with everyone in the project via the `.mcp.json` file
 - `user` : available to you across all projects. Older versions called this scope `global`
- Set environment variables with `--env` flags (for example, `--env KEY=value`)
- Configure MCP server startup timeout using the `MCP_TIMEOUT` environment variable (for example, `MCP_TIMEOUT=10000 claude` sets a 10-second timeout)
- Set a per-server tool execution timeout by adding a `timeout` field in milliseconds to that server's `.mcp.json` entry, for example `"timeout": 600000` for ten minutes. This overrides the `MCP_TOOL_TIMEOUT` environment variable for that server only
- Claude Code displays a warning when MCP tool output exceeds 10,000 tokens. To increase this limit, set the `MAX_MCP_OUTPUT_TOKENS` environment variable (for example, `MAX_MCP_OUTPUT_TOKENS=50000`)
- Use `/mcp` to authenticate with remote servers that require OAuth 2.0 authentication

The per-server `timeout` is a hard wall-clock limit per tool call, and progress notifications from the server don't extend it. Values below 1000 are ignored and fall through to `MCP_TOOL_TIMEOUT` , or to its default of about 28 hours when that variable is unset. Before v2.1.162, values below 1000 were floored to one second instead.

For HTTP and SSE servers, the per-request fetch first-byte budget has a 60-second minimum.

As of v2.1.187, a tool call to a remote HTTP, SSE, WebSocket, or [claude.ai connector](#) server that sends no response and no progress notification for 5 minutes aborts with an error instead of waiting for the wall-clock limit. Set the `CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT` environment variable in milliseconds to change the idle window, or set it to `0` to disable the check. Stdio servers are local processes and are not subject to the idle timeout.

Plugin-provided MCP servers

[Plugins](#) can bundle MCP servers, automatically providing tools and integrations when the plugin is enabled. Plugin MCP servers work identically to user-configured servers.

How plugin MCP servers work:

- Plugins define MCP servers in `.mcp.json` at the plugin root or inline in `plugin.json`
- When a plugin is enabled, its MCP servers start automatically
- Plugin MCP tools appear alongside manually configured MCP tools
- Plugin servers are managed through plugin installation, not `/mcp` commands

Example plugin MCP configuration:

In `.mcp.json` at plugin root:

```
{
  "mcpServers": {
    "database-tools": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/db-server",
      "args": ["--config", "${CLAUDE_PLUGIN_ROOT}/config.json"],
      "env": {
        "DB_URL": "${DB_URL}"
      }
    }
  }
}
```

Or inline in `plugin.json` :

```
{
  "name": "my-plugin",
  "mcpServers": {
    "plugin-api": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/api-server",
      "args": ["--port", "8080"]
    }
  }
}
```

Plugin MCP features:

- **Automatic lifecycle:** at session startup, servers for enabled plugins connect automatically. If you enable or disable a plugin during a session, run `/reload-plugins` to connect or disconnect its MCP servers
- **Environment variables:** use `${CLAUDE_PLUGIN_ROOT}` for bundled plugin files,

`${CLAUDE_PLUGIN_DATA}` for **persistent state** that survives plugin updates, and

`${CLAUDE_PROJECT_DIR}` for the stable project root

- **User environment access:** access to the same environment variables as manually configured servers
- **Multiple transport types:** support for stdio, SSE, HTTP, and WebSocket transports, though transport support may vary by server

Viewing plugin MCP servers:

```
# Within Claude Code, see all MCP servers including plugin ones  
/mcp
```

Plugin servers appear in the list with indicators showing they come from plugins.

Plugin MCP tool names:

Tools from a plugin-bundled MCP server include both the plugin name and the server key in their callable name. The full form is `mcp__plugin_<plugin-name>_<server-name>__<tool-name>`, where any character outside `A-Z`, `a-z`, `0-9`, `_`, and `-` is replaced with `_`. For the `database-tools` server bundled in a plugin named `my-plugin`, a `query` tool is callable as:

```
mcp__plugin_my-plugin_database-tools__query
```

Use this full name when referencing the tool in **permission rules**, a skill's `allowed-tools` list, or a **subagent's tools field**.

Benefits of plugin MCP servers:

- **Bundled distribution:** tools and servers packaged together
- **Automatic setup:** no manual MCP configuration needed
- **Team consistency:** everyone gets the same tools when the plugin is installed

See the **plugin components reference** for details on bundling MCP servers with plugins.

MCP installation scopes

MCP servers can be configured at three scopes. The scope you choose controls which projects the

server loads in and whether the configuration is shared with your team. Administrators can also deploy servers at the enterprise level via managed configuration.

Scope	Loads in	Shared with team	Stored in
<u>Local</u>	Current project only	No	<code>~/.claude.json</code>
<u>Project</u>	Current project only	Yes, via version control	<code>.mcp.json</code> in project root
<u>User</u>	All your projects	No	<code>~/.claude.json</code>

Local scope

Local scope is the default. A local-scoped server loads only in the project where you added it and stays private to you. Claude Code stores it in `~/.claude.json` under that project’s path, so the same server won’t appear in your other projects. Use local scope for personal development servers, experimental configurations, or servers with credentials you don’t want in version control.

❗ The term “local scope” for MCP servers differs from general local settings. MCP local-scoped servers are stored in `~/.claude.json` (your home directory), while general local settings use `.claude/settings.local.json` (in the project directory). See [Settings](#) for details on settings file locations.

```
# Add a local-scoped server (default)
claude mcp add --transport http stripe https://mcp.stripe.com

# Explicitly specify local scope
claude mcp add --transport http stripe --scope local
https://mcp.stripe.com
```

The command writes the server into the entry for your current project inside `~/.claude.json`. The example below shows the result when you run it from `/path/to/your/project` :

```
{
  "projects": {
    "/path/to/your/project": {
      "mcpServers": {
        "stripe": {
          "type": "http",
          "url": "https://mcp.stripe.com"
        }
      }
    }
  }
}
```

Project scope

Project-scoped servers enable team collaboration by storing configurations in a `.mcp.json` file at your project's root directory. This file is designed to be checked into version control, ensuring all team members have access to the same MCP tools and services. When you add a project-scoped server, Claude Code automatically creates or updates this file with the appropriate configuration structure.

```
# Add a project-scoped server
claude mcp add --transport http paypal --scope project
https://mcp.paypal.com/mcp
```

The resulting `.mcp.json` file follows a standardized format:

```
{
  "mcpServers": {
    "shared-server": {
      "command": "/path/to/server",
      "args": [],
      "env": {}
    }
  }
}
```

For security reasons, Claude Code prompts for approval before using project-scoped servers from `.mcp.json` files. If you need to reset these approval choices, use the `claude mcp reset-project-`

`choices` command.

User scope

User-scoped servers are stored in `~/.claude.json` and provide cross-project accessibility, making them available across all projects on your machine while remaining private to your user account. This scope works well for personal utility servers, development tools, or services you frequently use across different projects.

```
# Add a user server
claude mcp add --transport http hubspot --scope user
https://mcp.hubspot.com/anthropic
```

Scope hierarchy and precedence

When the same server is defined in more than one place, Claude Code connects to it once, using the definition from the highest-precedence source. The entire server entry from that source is used; fields are not merged across scopes.

1. Local scope
2. Project scope
3. User scope
4. Plugin-provided servers
5. claude.ai connectors

The three scopes match duplicates by name. Plugins and connectors match by endpoint, so one that points at the same URL or command as a server above is treated as a duplicate.

Environment variable expansion in `.mcp.json`

Claude Code supports environment variable expansion in `.mcp.json` files, allowing teams to share configurations while maintaining flexibility for machine-specific paths and sensitive values like API keys.

Supported syntax:

- `${VAR}` : expands to the value of environment variable `VAR`

- `${VAR:-default}` : expands to `VAR` if set, otherwise uses `default`

Expansion locations: Environment variables can be expanded in:

- `command` : the server executable path
- `args` : command-line arguments
- `env` : environment variables passed to the server
- `url` : for HTTP server types
- `headers` : for HTTP server authentication

Example with variable expansion:

```
{
  "mcpServers": {
    "api-server": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": {
        "Authorization": "Bearer ${API_KEY}"
      }
    }
  }
}
```

If a required environment variable isn't set and has no default value, Claude Code fails to parse the config.

Practical examples

Example: Monitor errors with Sentry

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

Authenticate with your Sentry account:

```
/mcp
```

Then debug production issues:

What are the most common errors in the last 24 hours?

Show me the stack trace for error ID abc123

Which deployment introduced these new errors?

Example: Connect to GitHub for code reviews

GitHub's remote MCP server authenticates with a GitHub personal access token passed as a header. To get one, open your **GitHub token settings**, generate a new fine-grained token with access to the repositories you want Claude to work with, then add the server:

```
claude mcp add --transport http github
https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

Then work with GitHub:

Review PR #456 and suggest improvements

Create a new issue for the bug we just found

Show me all open PRs assigned to me

Example: Query your PostgreSQL database

```
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub \
  --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"
```

Then query your database naturally:

```
What's our total revenue this month?
```

```
Show me the schema for the orders table
```

```
Find customers who haven't made a purchase in 90 days
```

Authenticate with remote MCP servers

Many cloud-based MCP servers require authentication. Claude Code supports OAuth 2.0 for secure connections.

Claude Code marks a remote server as needing authentication when the server responds with `401 Unauthorized` or `403 Forbidden`. Either status code flags the server in `/mcp` so you can complete the OAuth flow. A custom server that returns a `WWW-Authenticate` header pointing to its authorization server gets the same automatic discovery as any other remote server.

If you configured `headers.Authorization` for the server and the server rejects that header, Claude Code reports the connection as failed instead of falling back to OAuth. Check that the token is valid for the MCP endpoint, or remove the header to use the OAuth flow.

1 Add the server that requires authentication

For example:

```
claude mcp add --transport http sentry
https://mcp.sentry.dev/mcp
```

2 Use the `/mcp` command within Claude Code

In Claude Code, use the command:

/mcp

Then follow the steps in your browser to log in.

 Tips:

- Authentication tokens are stored securely and refreshed automatically
- Use “Clear authentication” in the `/mcp` menu to revoke access
- If your browser doesn’t open automatically, copy the provided URL and open it manually
- If the browser redirect fails with a connection error after authenticating, paste the full callback URL from your browser’s address bar into the URL prompt that appears in Claude Code
- OAuth authentication works with HTTP servers

Authenticate from the command line

From v2.1.186, `claude mcp login <name>` runs a configured server’s OAuth flow directly from your shell, so you don’t need to open the `/mcp` panel inside a session.

```
claude mcp login sentry
```

To clear stored credentials later, run `claude mcp logout <name>` .

As of v2.1.191, the command detects when no local browser is available, such as during an SSH session or on Linux without a display server, and prints the authorization URL instead of trying to open a browser. Open the URL on your local machine, then paste the full redirect URL from your browser’s address bar back at the prompt. The command needs an interactive terminal for the paste step, so connect with `ssh -t` . Pass `--no-browser` to force the URL prompt even when a local browser is detected.

```
claude mcp login sentry --no-browser
```

Use a fixed OAuth callback port

Some MCP servers require a specific redirect URI registered in advance. By default, Claude Code picks a random available port for the OAuth callback. Use `--callback-port` to fix the port so it

matches a pre-registered redirect URI of the form `http://localhost:PORT/callback` .

You can use `--callback-port` on its own (with dynamic client registration) or together with `--client-id` (with pre-configured credentials).

```
# Fixed callback port with dynamic client registration
claude mcp add --transport http \
  --callback-port 8080 \
  my-server https://mcp.example.com/mcp
```

Use pre-configured OAuth credentials

Some MCP servers don't support automatic OAuth setup via Dynamic Client Registration. If you see an error like "Incompatible auth server: does not support dynamic client registration," the server requires pre-configured credentials. Claude Code also supports servers that use a Client ID Metadata Document (CIMD) instead of Dynamic Client Registration, and discovers these automatically. If automatic discovery fails, register an OAuth app through the server's developer portal first, then provide the credentials when adding the server.

1 Register an OAuth app with the server

Create an app through the server's developer portal and note your client ID and client secret.

Many servers also require a redirect URI. If so, choose a port and register a redirect URI in the format `http://localhost:PORT/callback` . Use that same port with `--callback-port` in the next step.

2 Add the server with your credentials

Choose one of the following methods. The port used for `--callback-port` can be any available port. It needs to match the redirect URI you registered in the previous step.

```
claude mcp add
```

Use `--client-id` to pass your app's client ID. The `--client-secret` flag prompts for the secret with masked input:

```
claude mcp add --transport http \
  --client-id your-client-id --client-secret --callback-port
8080 \
  my-server https://mcp.example.com/mcp
```

claude mcp add-json

Include the `oauth` object in the JSON config and pass `--client-secret` as a separate flag:

```
claude mcp add-json my-server \

'{"type":"http","url":"https://mcp.example.com/mcp","oauth":
{"clientId":"your-client-id","callbackPort":8080}}' \
  --client-secret
```

claude mcp add-json (callback port only)

Use `--callback-port` without a client ID to fix the port while using dynamic client registration:

```
claude mcp add-json my-server \

'{"type":"http","url":"https://mcp.example.com/mcp","oauth":
{"callbackPort":8080}}'
```

CI / env var

Set the secret via environment variable to skip the interactive prompt:

```
MCP_CLIENT_SECRET=your-secret claude mcp add --transport
http \
  --client-id your-client-id --client-secret --callback-port
8080 \
  my-server https://mcp.example.com/mcp
```

3

Authenticate in Claude Code

Run `/mcp` in Claude Code and follow the browser login flow.

Tips:

- The client secret is stored securely in your system keychain (macOS) or a credentials file, not in your config
- If the server uses a public OAuth client with no secret, use only `--client-id` without `--client-secret`
- `--callback-port` can be used with or without `--client-id`
- These flags only apply to HTTP and SSE transports. They have no effect on stdio servers
- Use `claude mcp get <name>` to verify that OAuth credentials are configured for a server

Override OAuth metadata discovery

Point Claude Code at a specific OAuth authorization server metadata URL to bypass the default discovery chain. Set `authServerMetadataUrl` when the MCP server's standard endpoints error, or when you want to route discovery through an internal proxy. By default, Claude Code first checks RFC 9728 Protected Resource Metadata at `/.well-known/oauth-protected-resource`, then falls back to RFC 8414 authorization server metadata at `/.well-known/oauth-authorization-server`.

Set `authServerMetadataUrl` in the `oauth` object of your server's config in `.mcp.json`:

```
{
  "mcpServers": {
    "my-server": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "oauth": {
        "authServerMetadataUrl": "https://auth.example.com/.well-known/openid-configuration"
      }
    }
  }
}
```

The URL must use `https://`. `authServerMetadataUrl` requires Claude Code v2.1.64 or later. The metadata URL's `scopes_supported` overrides the scopes the upstream server advertises.

Restrict OAuth scopes

Set `oauth.scopes` to pin the scopes Claude Code requests during the authorization flow. This is the supported way to restrict an MCP server to a security-team-approved subset when the upstream authorization server advertises more scopes than you want to grant. The value is a single space-separated string, matching the `scope` parameter format in RFC 6749 §3.3.

```
{
  "mcpServers": {
    "slack": {
      "type": "http",
      "url": "https://mcp.slack.com/mcp",
      "oauth": {
        "scopes": "channels:read chat:write search:read"
      }
    }
  }
}
```

`oauth.scopes` takes precedence over both `authServerMetadataUrl` and the scopes the server discovers at `/.well-known`. Leave it unset to let the MCP server determine the requested scope set.

If the authorization server advertises `offline_access` in `scopes_supported`, Claude Code

appends it to the pinned scopes so the access token can be refreshed without a new browser sign-in.

If the server later returns a 403 `insufficient_scope` for a tool call, Claude Code re-authenticates with the same pinned scopes. Widen `oauth.scopes` when a tool you need requires a scope outside the pinned set.

Use dynamic headers for custom authentication

If your MCP server uses an authentication scheme other than OAuth, such as Kerberos, short-lived tokens, or an internal SSO, use `headersHelper` to generate request headers at connection time. Claude Code runs the command and merges its output into the connection headers.

```
{
  "mcpServers": {
    "internal-api": {
      "type": "http",
      "url": "https://mcp.internal.example.com",
      "headersHelper": "/opt/bin/get-mcp-auth-headers.sh"
    }
  }
}
```

The command can also be inline:

```
{
  "mcpServers": {
    "internal-api": {
      "type": "http",
      "url": "https://mcp.internal.example.com",
      "headersHelper": "echo '{\"Authorization\": \"Bearer \"$(get-token)\"}'"
    }
  }
}
```

Requirements:

- The command must write a JSON object of string key-value pairs to stdout

- The command runs in a shell with a 10-second timeout
- Dynamic headers override any static `headers` with the same name

The helper runs fresh on each connection, at session start and on reconnect. There is no caching, so your script is responsible for any token reuse.

Claude Code sets these environment variables when executing the helper:

Variable	Value
<code>CLAUDE_CODE_MCP_SERVER_NAME</code>	the name of the MCP server
<code>CLAUDE_CODE_MCP_SERVER_URL</code>	the URL of the MCP server

Use these to write a single helper script that serves multiple MCP servers.

 `headersHelper` executes arbitrary shell commands. When defined at project or local scope, it only runs after you accept the workspace trust dialog.

Add MCP servers from JSON configuration

If you have a JSON configuration for an MCP server, you can add it directly:

1

Add an MCP server from JSON

```
# Basic syntax
claude mcp add-json <name> '<json>'

# Example: Adding an HTTP server with JSON configuration
claude mcp add-json weather-api
'{"type":"http","url":"https://api.weather.com/mcp","headers":
{"Authorization":"Bearer token"}}'

# Example: Adding a stdio server with JSON configuration
claude mcp add-json local-weather
'{"type":"stdio","command":"/path/to/weather-cli","args":["-
-api-key","abc123"],"env":{"CACHE_DIR":"/tmp"}}'

# Example: Adding an HTTP server with pre-configured OAuth
credentials
claude mcp add-json my-server
'{"type":"http","url":"https://mcp.example.com/mcp","oauth":
{"clientId":"your-client-id","callbackPort":8080}}' --
client-secret
```

2

Verify the server was added

```
claude mcp get weather-api
```



Tips:

- Make sure the JSON is properly escaped in your shell
- The JSON must conform to the MCP server configuration schema
- You can use `--scope user` to add the server to your user configuration instead of the project-specific one

Import MCP servers from Claude Desktop

If you've already configured MCP servers in Claude Desktop, you can import them:

1 Import servers from Claude Desktop

```
# Basic syntax
claude mcp add-from-claude-desktop
```

2 Select which servers to import

After running the command, you'll see an interactive dialog that allows you to select which servers you want to import.

3 Verify the servers were imported

```
claude mcp list
```

Tips:

- This feature only works on macOS and Windows Subsystem for Linux (WSL)
- It reads the Claude Desktop configuration file from its standard location on those platforms
- Use the `--scope user` flag to add servers to your user configuration
- Imported servers keep the same names as in Claude Desktop
- If servers with the same names already exist, they get a numerical suffix (for example, `server_1`)

Use MCP servers from claude.ai

If you've logged into Claude Code with a [claude.ai](#) account, MCP servers you've added in claude.ai are automatically available in Claude Code:

1 Configure MCP servers in claude.ai

Add servers at [claude.ai/customize/connectors](#). On Team and Enterprise plans, only admins can add servers.

2 Authenticate the MCP server

Complete any required authentication steps in claude.ai.

3 View and manage servers in Claude Code

In Claude Code, use the command:

```
/mcp
```

Servers from claude.ai appear in the list with indicators showing they come from claude.ai.

From v2.1.161, connectors you have never signed in to are collapsed behind a `Show unused connectors` row at the end of the claude.ai section, so an organization-provisioned list doesn't fill the panel. Select the row to expand them. A connector you signed in to before stays visible even when it currently needs re-authentication.

Connectors from claude.ai are fetched only when your active **authentication method** is your claude.ai subscription. They aren't loaded when `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, `apiKeyHelper`, or a third-party provider such as Bedrock or Vertex is active, even if you previously ran `/login`.

If `/mcp` doesn't list a connector you added, run `/status` to confirm which authentication method is active, unset that environment variable or remove the `apiKeyHelper` setting, then run `/login` to select your claude.ai account.

A server you've added in Claude Code takes **precedence** over a claude.ai connector that points at the same URL. When this happens, `/mcp` lists the connector as hidden and shows how to remove the duplicate if you'd rather use the connector.

Some Anthropic-hosted connectors, such as Microsoft 365, Gmail, and Google Calendar, don't support local OAuth from Claude Code because the upstream identity provider only accepts the redirect URL that claude.ai registered. From v2.1.162, authenticating one of these hosts in `/mcp` shows a message directing you to connect it at Settings → Connectors on claude.ai instead. Once connected there, the connector appears in Claude Code automatically.

Disable claude.ai connectors

To disable claude.ai MCP servers in Claude Code, set **disableClaudeAiConnectors** to `true` in any settings scope:

```
{  
  "disableClaudeAiConnectors": true  
}
```

This setting uses any-source-true semantics: `true` in any settings source takes precedence. A checked-in project `.claude/settings.json` can opt a repository out of cloud connectors, but a project-level `false` can't re-enable connectors that a user- or policy-level `true` has disabled. Servers passed explicitly via `--mcp-config` are unaffected.

You can also set the `ENABLE_CLAUDEAI_MCP_SERVERS` environment variable to `false`, which has the same effect for the current shell session:

```
ENABLE_CLAUDEAI_MCP_SERVERS=false claude
```

To block individual claude.ai connectors instead of all of them, add them to `deniedMcpServers` by name or by URL pattern. For example, a `serverName` entry of `"claude.ai Slack"` blocks the Slack connector. To toggle a connector on or off for the current project only, use the `/mcp` panel.

ⓘ These client-side settings govern local Claude Code sessions. In [Claude Code on the web](#) sessions, claude.ai connectors are provisioned by the remote host and arrive as explicit `--mcp-config` entries, so `disableClaudeAiConnectors` doesn't apply there. Connector URLs are also rewritten through the session proxy, so a `deniedMcpServers` `serverUrl` pattern targeting the vendor URL won't match. Manage which connectors a cloud session can use from your claude.ai organization settings.

Use Claude Code as an MCP server

You can use Claude Code itself as an MCP server that other applications can connect to:

```
# Start Claude as a stdio MCP server  
claude mcp serve
```

You can use this in Claude Desktop by adding this configuration to `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "claude-code": {
      "type": "stdio",
      "command": "claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

⚠ **Configuring the executable path:** the `command` field must reference the Claude Code executable. If the `claude` command is not in your system's PATH, you'll need to specify the full path to the executable.

To find the full path:

```
which claude
```

Then use the full path in your configuration:

```
{
  "mcpServers": {
    "claude-code": {
      "type": "stdio",
      "command": "/full/path/to/claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

Without the correct executable path, you'll encounter errors like `spawn claude ENOENT`.

Tips:

- The server provides access to Claude's tools like View, Edit, LS, etc.
- In Claude Desktop, try asking Claude to read files in a directory, make edits, and more.
- This MCP server only exposes Claude Code's tools to your MCP client, so your own client is responsible for implementing user confirmation for individual tool calls.

MCP output limits and warnings

When MCP tools produce large outputs, Claude Code helps manage the token usage to prevent overwhelming your conversation context:

- **Output warning threshold:** Claude Code displays a warning when any MCP tool output exceeds 10,000 tokens
- **Configurable limit:** you can adjust the maximum allowed MCP output tokens using the `MAX_MCP_OUTPUT_TOKENS` environment variable
- **Default limit:** the default maximum is 25,000 tokens
- **Scope:** the environment variable applies to tools that don't declare their own limit. Tools that set `anthropic/maxResultSizeChars` use that value instead for text content, regardless of what `MAX_MCP_OUTPUT_TOKENS` is set to. Tools that return image data are still subject to `MAX_MCP_OUTPUT_TOKENS`

To increase the limit for tools that produce large outputs:

```
export MAX_MCP_OUTPUT_TOKENS=50000
claude
```

This is particularly useful when working with MCP servers that:

- Query large datasets or databases
- Generate detailed reports or documentation
- Process extensive log files or debugging information

Raise the limit for a specific tool

If you're building an MCP server, you can allow individual tools to return results larger than the

default persist-to-disk threshold by setting `_meta["anthropic/maxResultSizeChars"]` in the tool's `tools/list` response entry. Claude Code raises that tool's threshold to the annotated value, up to a hard ceiling of 500,000 characters.

This is useful for tools that return inherently large but necessary outputs, such as database schemas or full file trees. Without the annotation, results that exceed the default threshold are persisted to disk and replaced with a file reference in the conversation.

```
{
  "name": "get_schema",
  "description": "Returns the full database schema",
  "_meta": {
    "anthropic/maxResultSizeChars": 200000
  }
}
```

The annotation applies independently of `MAX_MCP_OUTPUT_TOKENS` for text content, so users don't need to raise the environment variable for tools that declare it. Tools that return image data are still subject to the token limit.

 If you frequently encounter output warnings with specific MCP servers you don't control, consider increasing the `MAX_MCP_OUTPUT_TOKENS` limit. You can also ask the server author to add the `anthropic/maxResultSizeChars` annotation or to paginate their responses. The annotation has no effect on tools that return image content; for those, raising `MAX_MCP_OUTPUT_TOKENS` is the only option.

Respond to MCP elicitation requests

MCP servers can request structured input from you mid-task using elicitation. When a server needs information it can't get on its own, Claude Code displays an interactive dialog and passes your response back to the server. No configuration is required on your side: elicitation dialogs appear automatically when a server requests them.

Servers can request input in two ways:

- **Form mode:** Claude Code shows a dialog with form fields defined by the server (for example, a username and password prompt). Fill in the fields and submit.
- **URL mode:** Claude Code opens a browser URL for authentication or approval. Complete the flow in the browser, then confirm in the CLI.

To auto-respond to elicitation requests without showing a dialog, use the [Elicitation hook](#).

If you're building an MCP server that uses elicitation, see the [MCP elicitation specification](#) for protocol details and schema examples.

Use MCP resources

MCP servers can expose resources that you can reference using @ mentions, similar to how you reference files.

Reference MCP resources

1 List available resources

Type @ in your prompt to see available resources from all connected MCP servers. Resources appear alongside files in the autocomplete menu.

2 Reference a specific resource

Use the format `@server:protocol://resource/path` to reference a resource:

Can you analyze @github:issue://123 and suggest a fix?

Please review the API documentation at
@docs:file://api/authentication

3 Multiple resource references

You can reference multiple resources in a single prompt:

Compare @postgres:schema://users with
@docs:file://database/user-model

Tips:

- Resources are automatically fetched and included as attachments when referenced
- Resource paths are fuzzy-searchable in the @ mention autocomplete
- Claude Code automatically provides tools to list and read MCP resources when servers support them
- Resources can contain any type of content that the MCP server provides (text, JSON, structured data, etc.)

Scale with MCP tool search

Tool search keeps MCP context usage low by deferring tool definitions until Claude needs them. Only tool names and server instructions load at session start, so adding more MCP servers has minimal impact on your context window. Claude Code doesn't impose a fixed per-server tool cap; the practical limit is your context window budget.

How it works

Tool search is enabled by default. MCP tools are deferred rather than loaded into context upfront, and Claude uses a search tool to discover relevant ones when a task needs them. Only the tools Claude actually uses enter context. From your perspective, MCP tools work exactly as before.

If you prefer threshold-based loading, set `ENABLE_TOOL_SEARCH=auto` to load schemas upfront when they fit within 10% of the context window and defer only the overflow. See [Configure tool search](#) for all options.

For MCP server authors

If you're building an MCP server, the server instructions field becomes more useful with tool search enabled. Server instructions help Claude understand when to search for your tools, similar to how [skills](#) work.

Add clear, descriptive server instructions that explain:

- What category of tasks your tools handle
- When Claude should search for your tools
- Key capabilities your server provides

Claude Code truncates tool descriptions and server instructions at 2KB each. Keep them concise to avoid truncation, and put critical details near the start.

Configure tool search

Tool search is enabled by default: MCP tools are deferred and discovered on demand. Claude Code disables it by default on Vertex AI. It is also disabled when `ANTHROPIC_BASE_URL` points to a non-first-party host, since most proxies don't forward `tool_reference` blocks. Set `ENABLE_TOOL_SEARCH` explicitly to override either fallback.

Tool search requires a model that supports `tool_reference` blocks. Haiku models don't support it. On Vertex AI, tool search is supported for Claude Sonnet 4.5 and later and Claude Opus 4.5 and later.

Control tool search behavior with the `ENABLE_TOOL_SEARCH` environment variable:

Value	Behavior
(unset)	All MCP tools deferred and loaded on demand. Falls back to loading upfront on Vertex AI or when <code>ANTHROPIC_BASE_URL</code> is a non-first-party host
<code>true</code>	All MCP tools deferred. Claude Code sends the beta header even on Vertex AI and through proxies. Requests fail on Vertex AI models earlier than Sonnet 4.5 or Opus 4.5, or on proxies that don't support <code>tool_reference</code> blocks
<code>auto</code>	Threshold mode: tools load upfront if they fit within 10% of the context window, deferred otherwise
<code>auto:N</code>	Threshold mode with a custom percentage, where <code>N</code> is 0-100. For example, <code>auto:5</code> for 5%
<code>false</code>	All MCP tools loaded upfront, no deferral

```
# Use a custom 5% threshold
ENABLE_TOOL_SEARCH=auto:5 claude

# Disable tool search entirely
ENABLE_TOOL_SEARCH=false claude
```

Or set the value in your `settings.json` env field.

You can also disable the `ToolSearch` tool specifically:


```
{
  "permissions": {
    "deny": ["ToolSearch"]
  }
}
```

Exempt a server from deferral

If a server's tools should always be visible to Claude without a search step, set `alwaysLoad` to `true` in that server's configuration. Every tool from that server then loads into context at session start regardless of the `ENABLE_TOOL_SEARCH` setting. Use this for a small number of tools that Claude needs on every turn, since each upfront tool consumes context that would otherwise be available for your conversation.

The following `.mcp.json` entry exempts one HTTP server while leaving other servers deferred:

```
{
  "mcpServers": {
    "core-tools": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "alwaysLoad": true
    }
  }
}
```

The `alwaysLoad` field is available on all server types and requires Claude Code v2.1.121 or later. An MCP server can also mark individual tools as always-loaded by including `"anthropic/alwaysLoad": true` in the tool's `_meta` object, which has the same effect for that tool only.

Setting `alwaysLoad: true` also blocks startup until the server connects, capped at the standard 5-second connect timeout. This applies even though MCP startup is otherwise **non-blocking by default**, since the tools must be present when the first prompt is built. Other servers continue to connect in the background.

Use MCP prompts as commands

MCP servers can expose prompts that become available as commands in Claude Code.

Execute MCP prompts

1 Discover available prompts

Type `/` to see all available commands, including those from MCP servers. MCP prompts appear with the format `/mcp__servername__promptname`.

2 Execute a prompt without arguments

```
/mcp__github__list_prs
```

3 Execute a prompt with arguments

Many prompts accept arguments. Pass them space-separated after the command:

```
/mcp__github__pr_review 456
```

```
/mcp__jira__create_issue "Bug in login flow" high
```

Tips:

- MCP prompts are dynamically discovered from connected servers
- Arguments are parsed based on the prompt's defined parameters
- Prompt results are injected directly into the conversation
- Server and prompt names are normalized, with spaces converted to underscores

Managed MCP configuration

For organizations that need centralized control over which MCP servers users can connect to, see [Managed MCP configuration](#). It covers deploying a fixed server set with `managed-mcp.json`, restricting servers with `allowedMcpServers` and `deniedMcpServers`, and what users see when a server is blocked.

